

Informe LAB 03

Santiago Murillo Ibañez
Jorge Esteban Chaves Chacon

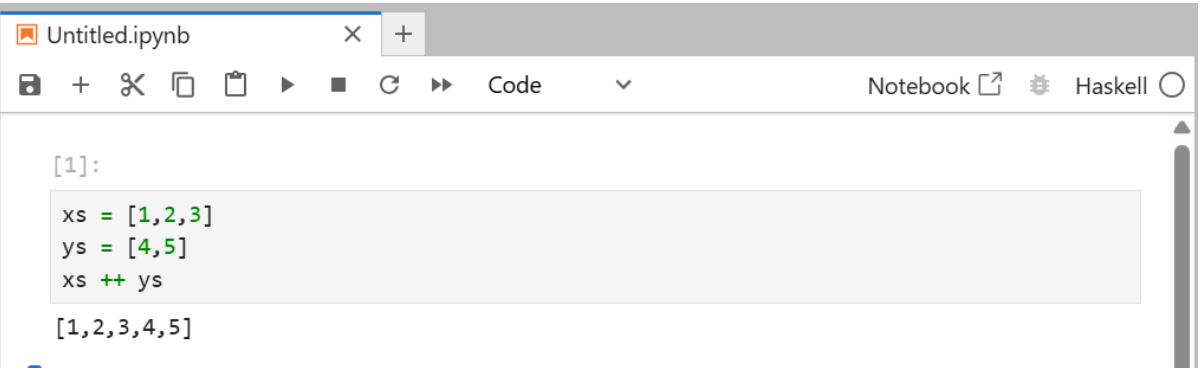
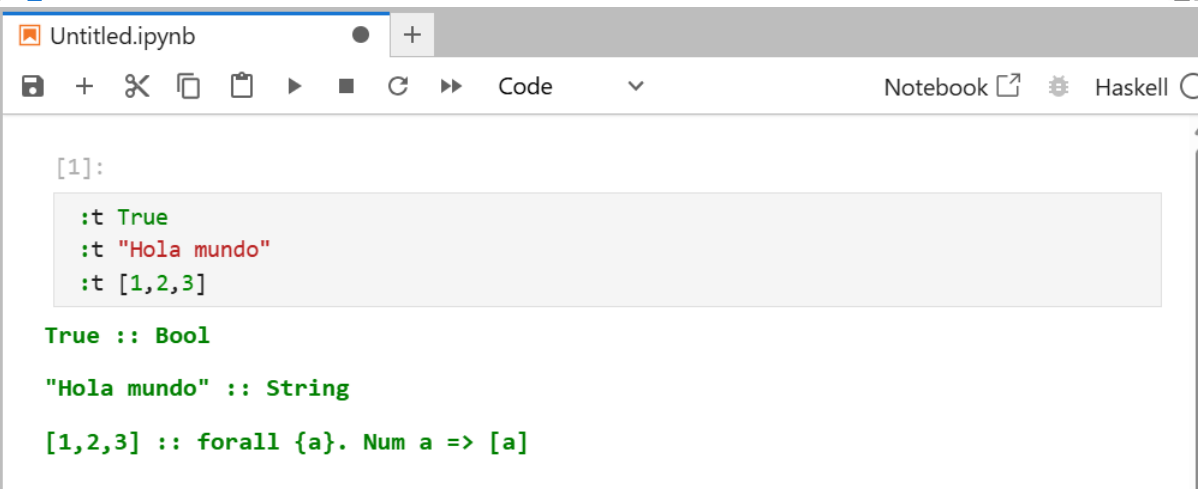
Universidad Escuela Colombiana de Ingeniería Julio Garavito

Paradigmas de la Programación

Juan Camilo Ángel Hernández

14 de septiembre de 2026

Ejercicio 1 - Fundamentos básicos y tipos

- 1) The screenshot shows a Jupyter Notebook interface with a single code cell. The code defines two lists, `xs` and `ys`, and concatenates them using `xs ++ ys`. The output is the resulting list `[1,2,3,4,5]`.
- ```
[1]:
xs = [1,2,3]
ys = [4,5]
xs ++ ys
[1,2,3,4,5]
```
- 2) The screenshot shows a Jupyter Notebook interface with a single code cell. The code defines three variables: `t` with the value `True`, `t` with the value `"Hola mundo"`, and `t` with the value `[1,2,3]`. The output shows the values and their corresponding Haskell types: `True :: Bool`, `"Hola mundo" :: String`, and `[1,2,3] :: forall {a}. Num a => [a]`.
- ```
[1]:
:t True
:t "Hola mundo"
:t [1,2,3]

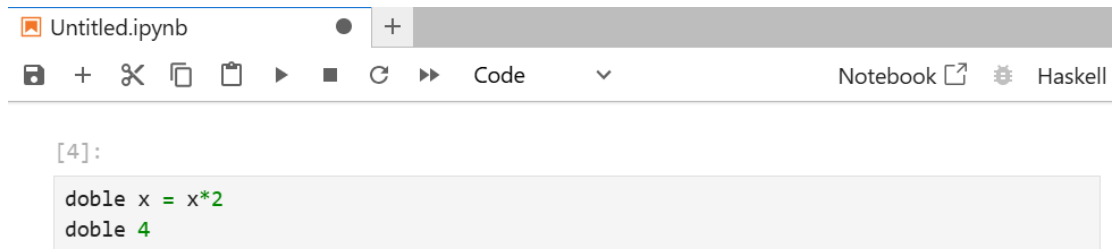
True :: Bool
"Hola mundo" :: String
[1,2,3] :: forall {a}. Num a => [a]
```

¿Qué significa que Haskell sea un lenguaje fuertemente tipado y evaluado perezosamente?

Que sea fuertemente tipado significa que cada variable antes de instanciarla y durante su ejecución solo puede pertenecer a un tipo y esto no puede cambiar en ningún momento.

Que sea perezoso significa que no ejecuta funciones ni calcula resultados hasta que se ve forzado a hacerlo

3)

The screenshot shows a Jupyter Notebook interface with a single code cell. The code defines a variable `x` with the value `4` and then multiplies it by 2 using `x*2`. The output is the result of the multiplication, `8`.

```
[4]:
doble x = x*2
doble 4
```

a) 8

b)

```
[5]:
esMayorEdad :: Int -> Bool
esMayorEdad edad = edad >= 18
esMayorEdad 15

False
```

c)

```
[9]:
areaCirculo :: Float -> Float
areaCirculo radio = (radio*radio)*pi
areaCirculo 5.0

78.53982
```

¿Qué diferencias encuentras entre los tipos Int y Float?

Int nos da números enteros de máximo 64 bits, mientras que float son los números reales de máxima precisión 32 bits.

Ejercicio 2 - Listas y operaciones básicas

1)

```
[10]:
head [10,20,30,40]
tail [10,20,30,40]
take 3 [1..10]
drop 4 [1..10]
length [1,2,3,4,5]

10
[20,30,40]
[1,2,3]
[5,6,7,8,9,10]
5
```

¿Qué hace la expresión [1..10] y cómo se relaciona con la evaluación perezosa?

La lista de uno a diez se va generando en memoria en la manera que se necesita por lo tanto se crea un **thunk** (una promesa de evaluación) y la lista se genera por elementos hasta cuando drop le indique

2) A

a)

```
[16]:
primerElemento :: [a] -> a
primerElemento (x:xs) = x
lista = [1..10]
primerElemento lista

1
```

b)

```
[19]:
productoLista :: [Int] -> Int
productoLista [] = 1
productoLista (x:xs) = x * productoLista xs
lista = [1..5]
productoLista lista

Use foldr
Found:
productoLista [] = 1
productoLista (x : xs) = x * productoLista xs
Why Not:
productoLista xs = foldr (*) 1 xs

120
```

c)

```
[21]:
eliminarNegativos :: [Int] -> [Int]
eliminarNegativos xs = [x | x <- xs, x >= 0]
eliminarNegativos [-10..10]

[0,1,2,3,4,5,6,7,8,9,10]
```

Untitled.ipynb

[2]:

```
invertir :: [a] -> [a]
invertir [] = []
invertir (x:xs) = invertir(xs) ++ [x]
invertir [1..10]
```

Redundant bracket
Found:Why Not:
 (xs) xs
 [10,9,8,7,6,5,4,3,2,1]

d)

Ejercicio 3 - Definición de funciones: composición y casos

1) A

Untitled.ipynb

[4]:

```
sumaDosNumeros :: Num a => a -> a -> a
sumaDosNumeros x y = x + y
cuadrado :: Int -> Int
cuadrado x = x*x
sumaCuadrados :: Int -> Int -> Int
sumaCuadrados x y = sumaDosNumeros (cuadrado x) (cuadrado y)
sumaCuadrados 4 3
```

25

a)

Untitled.ipynb

[8]:

```
duplicar :: Int -> Int
duplicar x = 2*x
dobleLista :: [Int] -> [Int]
dobleLista xs = [duplicar x | x <- xs]
dobleLista [1..5]
```

[2,4,6,8,10]

b)

La forma más simple de hacer dobleLista es utilizar el map

```
Untitled.ipynb
+
[9]:
doblelista :: [Int] -> [Int]
doblelista = map duplicar
doblelista [1..5]

[2,4,6,8,10]
```

2)

```
Untitled.ipynb
+
[12]:
clasificarNota :: Int -> String
clasificarNota n
  | n < 3    = "Reprobado"
  | n < 4    = "En recuperacion"
  | otherwise = "Aprobado"
clasificarNota 0
clasificarNota 3
clasificarNota 4
clasificarNota 4

"Reprobado"
"En recuperacion"
"Aprobado"
"Aprobado"
```

a)

```
Untitled.ipynb
+
[1]:
fibonacci :: Int -> Int
fibonacci 0 = 1
fibonacci 1 = 1
fibonacci n = fibonacci (n-1) + fibonacci (n-2)
fibonacci 5

8
```

b)

Ejercicio 4 - Evaluación funcional

1)

a) Va a quedar una lista de longitud cinco que contenga [2,4,6,8,10]

```
[3]:
take 5 (map (*2) [1..])

b) [2,4,6,8,10]
```

¿Por qué [1..] no causa un ciclo infinito?

Debido a que haskell es perezoso solo necesito cargar una lista de cinco posiciones y no necesito guardar en memoria una lista infinita

2) misterio x y = if x > 10 then y * 2
 else x + y

- a) Miramos que nuestra x es (5+7) , y es (4*3)
- b) Como x(12) es mayor a 10, vamos a ir por la primera guarda
- c) Como y es 12 vamos a multiplicarla por dos y ese sera nuestro resultado
- d) 24

```
[4]:
misterio x y = if x > 10 then y * 2 else x + y
misterio (5+7) (4*3)

24
```

Ejercicio 5 - Expresiones Lambda

1. Define las siguientes funciones lambda

a. Reciba un número y retorne su doble.

```
[17]: (\x -> x * 2) 7

Avoid lambda using `infix`
Found:      Why Not:
(\ x -> x * 2)(* 2)
14
```

b. Calcular el cubo de un número

```
Untitled.ipynb
+
Code
Notebook Haskell

[18]: (\x -> x / 2) 9

Avoid lambda using `infix`
Found:      Why Not:
(\ x -> x / 2)(/ 2)
4.5
```

c. Dividir un número entre 2

```
Untitled.ipynb
+
Code
Notebook Haskell

[19]: (\x -> x > 0) (-4)

Avoid lambda using `infix`
Found:      Why Not:
(\ x -> x > 0)(> 0)
False
```

d. Determinar si un número es positivo

```
Untitled.ipynb
+
Code
Notebook Haskell

[21]: (\x -> x > 0) (-4)

Avoid lambda using `infix`
Found:      Why Not:
(\ x -> x > 0)(> 0)
False
```

e. Determinar si un número pertenece al intervalo [10, 20]

```
Untitled.ipynb
+
Code
Notebook Haskell

[20]: (\x -> x >= 10 && x <= 20) 15

True
```

2. Define las siguientes funciones lambda de dos argumentos

a) La suma de dos números

```
Untitled.ipynb
+
Code
Notebook Haskell

[22]: (\x y -> x + y) 3 4

Avoid lambda
Found:      Why Not:
\ x y -> x + y(+)
7
```

b) La multiplicación de dos números

```
Untitled.ipynb
+
Code
Notebook Haskell

[23]: (\x y -> x * y) 6 7

Avoid lambda
Found:      Why Not:
\ x y -> x * y(*)
42
```

c) La potencia entre dos números



Untitled.ipynb

[24]: `(\x y -> x ^ y) 2 10`

Avoid lambda
Found: `\ x y -> x ^ y`
Why Not: `1024`

d) El máximo entre dos números



Untitled.ipynb

[25]: `(\x y -> if x > y then x else y) 9 4`

Use max
Found: `if x > y then x else y`
Why Not: `max x y`
`9`

e) Una función que determine si dos números son iguales



Untitled.ipynb

[26]: `(\x y -> x == y) 5 5`

Avoid lambda
Found: `\ x y -> x == y`
Why Not: `True`

Retrospectiva

1. **¿Cuál fue el tiempo total invertido en el laboratorio por cada uno de ustedes?** (Horas/Hombre)

Santiago Murillo 2 horas

Esteban Chaves 1 ½ horas

2. **¿Cuál es el estado actual del laboratorio? ¿Por qué?**

El laboratorio se encuentra terminado. Se resolvieron los cinco ejercicios propuestos (fundamentos y tipos, listas y operaciones básicas, definición de funciones con composición y casos, evaluación funcional y expresiones lambda), junto con todas las preguntas teóricas y de reflexión intercaladas. Cada función fue implementada en el notebook de Haskell, ejecutada con casos de prueba propios y documentada mediante los pantallazos correspondientes en este informe. No quedaron puntos pendientes ni funciones sin verificar.

3. **¿Cuál consideran fue el mayor logro? ¿Por qué?**

Comprender de manera práctica la “evaluación perezosa” de Haskell. El punto de quiebre fue el Ejercicio 4, con la expresión “take 5 (map (*2) [1..])”. Intuitivamente uno espera que una lista infinita cuelgue el programa, y verificar que devuelve `[2,4,6,8,10]` sin problema obligó a entender que Haskell trabaja con “thunks” y solo calcula los elementos que efectivamente se consumen. Ese concepto no tiene equivalente directo en los lenguajes imperativos que veníamos usando, y entenderlo cambió la forma de leer el resto del código del laboratorio.

4. **¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?**

El mayor obstáculo fue el “sistema de tipos”, más estricto que el de los lenguajes usados previamente. Concretamente, Haskell no realiza conversión implícita entre ‘Int’ y ‘Float’, por lo que operaciones como la división en ‘mitadL’ o el cálculo en ‘areaCirculo’ fallaban al mezclar tipos numéricos. Se resolvió revisando las firmas con ‘:t’ antes de escribir cada función y declarando explícitamente el tipo de cada una, lo que permitió que el compilador señalara la incompatibilidad antes de ejecutar.

Un segundo inconveniente fueron las sugerencias de HLINT que aparecen en el notebook. Al principio se interpretaron como errores; tras revisarlas se confirmó que son recomendaciones de estilo y no fallas de compilación, y se conservó la versión recursiva explícita porque el enunciado pedía justamente resolverlo de forma recursiva.

5. **¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?**

Se hizo bien la verificación individual de cada función antes de documentarla: ninguna implementación se dio por correcta sin ejecutarla con al menos un caso de prueba, lo que evitó incluir código con errores en el informe. También se mantuvo un orden consistente en el notebook, con una celda por punto del enunciado, lo que facilitó armar el documento de entrega.

Compromisos de mejora:

- Escribir siempre la firma de tipo explícita antes del cuerpo de la función, incluso cuando Haskell puede inferirla, para detectar errores de tipo más temprano.
- Probar cada función con casos límite y no solo con el caso feliz.
 - Atender las sugerencias de HLint y, cuando se decida no aplicarlas, dejar constancia del motivo.

6. **¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.**

- Lipovača, M. (2011). Learn You a Haskell for Great Good!: A Beginner's Guide.
- Haskell.org. (s.f.). Haskell Documentation.
- O'Sullivan, B., Goerzen, J., & Stewart, D. (2008). Real World Haskell. O'Reilly
- Hoogle. (s.f.). Haskell API search engine.

La más útil fue “Learn You a Haskell for Great Good!”, específicamente los capítulos sobre sintaxis de funciones y listas, porque explica el pattern matching, las guardas y la evaluación perezosa con ejemplos ejecutables muy cercanos a los pedidos en este laboratorio.